

GitHub-First Architecture for a Competitive Exam Image & Application Toolkit

Detailed technical and product architecture for a low-cost, privacy-focused web platform

1. Project Vision

The proposed platform is a web-based Competitive Exam Application Toolkit designed to help students and applicants prepare photographs, signatures, thumb impressions, handwritten declarations, documents and PDFs according to exam-specific upload requirements. The first version should be GitHub-first: source code, exam specifications and static assets are maintained in GitHub, while GitHub Pages hosts the website.

2. Core Product Concept

The primary user flow is:

1. Select examination
2. Select document type (Photo / Signature / Thumb / Declaration / Document)
3. Upload image
4. Automatically crop and resize
5. Automatically optimize file size
6. **Background Cleanup**
7. Validate dimensions, format and KB range
8. Show PASS / NEEDS ADJUSTMENT status
9. Download the final file

3. GitHub-First Architecture

Recommended initial architecture:

Component	Purpose
GitHub Repository	Source code, exam specification data, documentation and assets
GitHub Pages	Static website hosting
Browser	Local image crop, resize, compression and validation
JSON files	Exam-specific requirements
GitHub Actions	Automated testing/build/deployment where useful
Optional backend later	Authentication, cloud storage, database, payments

4. Recommended Repository Structure

```
exam-application-toolkit/
├── index.html
├── README.md
├── LICENSE
├── css/
│   └── style.css
├── js/
│   ├── app.js
│   ├── image-resizer.js
│   ├── image-cropper.js
│   ├── compressor.js
│   ├── validator.js
│   └── ui.js
├── data/
│   ├── exams.json
│   ├── categories.json
│   └── sources.json
├── assets/
│   ├── icons/
│   ├── images/
│   └── logos/
├── tests/
│   └── validator-tests.js
└── .github/
    ├── workflows/
    │   └── deploy.yml
```

5. Exam Specification Database

The exam database should be separated from the image-processing code. This allows requirements to be updated without rewriting the application.

```
{
  "exam": "IBPS PO",
  "year": 2026,
  "photo": {
    "width": 200,
    "height": 230,
    "minKB": 20,
    "maxKB": 50,
    "format": "JPG"
  },
  "signature": {
    "width": 140,
```

```
    "height": 60,  
    "minKB": 10,  
    "maxKB": 20,  
    "format": "JPG"  
  },  
  "sourceType": "Official notification",  
  "verifiedDate": "YYYY-MM-DD"  
}
```

6. Official-Source Verification

Exam requirements change between recruitment cycles and sometimes differ between exams conducted by the same organization. Therefore, the database should store the exam, recruitment/year, individual file requirements, source type, official notification reference and verification date.

- Never treat an old third-party specification as the current requirement.
- Prefer the official examination/recruitment notification.
- Display the notification year and last verified date.
- Where practical, provide a link to the official notification.
- Keep historical specifications rather than overwriting them.

7. Browser-Based Image Processing

The strongest feature of the GitHub-first approach is that ordinary image operations can occur entirely in the user's browser. The initial platform therefore does not need to upload photographs or signatures to a server.

- Read image locally
- Detect or allow manual crop
- Resize to required pixel dimensions
- Convert to required format
- Adjust JPEG/WebP quality as appropriate
- Iteratively compress to meet the KB range
- Validate final dimensions and file size
- Generate downloadable output

8. Privacy-by-Design

Because applicants may upload personal photographs, signatures, thumb impressions and identity-related documents, local browser processing should be the default. Images need not be stored on the server.

Recommended user-facing message:

“Your images are processed on your device and are not uploaded to our server.”

If a future feature requires server-side processing, the privacy policy should clearly state what is uploaded, why it is required, how long it is retained and how users can delete it.

9. Main Features

Feature	Purpose
Exam-specific photo resizer	Automatically applies the selected exam's dimensions and KB range.
Signature resizer	Crop, resize, compress and validate signatures.
Thumb impression tool	Prepare thumb impressions according to the selected specification.
Handwritten declaration tool	Crop, resize, compress and validate declarations.
Automatic KB optimizer	Adjust compression until the output falls within the specified range.
Manual crop controls	Allow the user to correct automatic cropping.
Face-aware crop	Optional advanced feature; should preserve natural appearance and follow exam rules.
Upload requirement checker	Shows format, dimensions and file-size validation results.
Image-to-PDF / PDF compressor	Future expansion of the toolkit.
Exam checklist	Future feature showing all required uploads and preparation steps.

10. Validation Engine

Every processed file should pass through a validation layer before download:

- File format
- Width in pixels
- Height in pixels
- Aspect ratio
- Minimum file size
- Maximum file size
- Basic readability/quality checks where feasible
- Exam-specific rules

Example result:

✓ Format correct ✓ Dimensions correct ✓ File size correct ✓ Ready to upload

11. User Interface

Recommended simple interface:

Competitive Exam Application Toolkit

[Select Examination ▼]

Document Type

[Photo] [Signature] [Thumb] [Declaration]

[Upload Image]

Preview

[Crop / Adjust]

Required: 200 × 230 px | 20–50 KB | JPG

[Process Image]

Validation

✓ Dimensions

✓ File size

✓ Format

[DOWNLOAD]

[Check another document]

12. GitHub Pages Deployment

1. Create a GitHub repository.
2. Add the static website files.
3. Push the project to the main branch.
4. Enable GitHub Pages in repository settings.
5. Choose the appropriate deployment source.
6. Test the published website.
7. Optionally connect a custom domain.
8. Use HTTPS through GitHub Pages.

13. Development Workflow

A simple GitHub workflow is sufficient for the first version:

9. Create feature branch
10. Develop/test locally
11. Commit changes
12. Open pull request
13. Review/test
14. Merge into main
15. Deploy automatically

14. What GitHub Provides vs. What Requires Additional Services

Requirement	GitHub-first solution	Later option
Source code	GitHub repository	GitHub
Website hosting	GitHub Pages	Cloudflare/Vercel/etc.
Exam data	JSON in repository	Supabase/PostgreSQL
Image processing	Browser	Server/API only if needed
Authentication	Not needed initially	Supabase/Firebase/Auth service
Payments	Not supported natively	Razorpay/Stripe
Cloud image storage	Not recommended initially	Object storage if required
Advanced AI	Not appropriate for core processing	External AI/API service

15. Cost Estimate

Item	Initial cost	Notes
GitHub	₹0	Free tier can support the initial project.
GitHub Pages	₹0	Suitable for static hosting.
Browser image processing	₹0	Runs on user's device.
Database	₹0 initially	Use JSON files first.
SSL/HTTPS	₹0	Provided with GitHub Pages.
Domain	Approx. ₹800–₹1,500/year	Depends on domain extension/registrar.
Advanced backend	Later	Only when required.

16. Product Roadmap

Version	Scope
Version 1	Photo + Signature resize, crop, compression and validation.
Version 2	Exam-specific presets and official-source database.
Version 3	Thumb impression, handwritten declaration and PDF tools.
Version 4	Exam application checklist and notification-linked requirements.
Version 5	Android app and advanced document tools.
Version 6	Optional accounts, premium tools, analytics and payment integration.

17. Suggested Long-Term Product

Rather than positioning the service only as an image resizer, the long-term product can become a Competitive Exam Application Toolkit:

- Photo Resizer
- Signature Resizer
- Thumb Impression Resizer
- Handwritten Declaration Resizer
- PDF Compressor
- Image-to-PDF Converter
- Upload Requirement Checker
- Exam-wise Application Checklist
- Exam Notification Tracker
- Preparation Resources

18. Recommended Technology Stack

Layer	Recommendation
Frontend	HTML/CSS/JavaScript initially; React/Next.js when the UI grows
Image processing	HTML5 Canvas and JavaScript; Web

	Workers/WebAssembly if performance requires
Data	JSON initially; PostgreSQL/Supabase later
Hosting	GitHub Pages initially
Version control	GitHub
Automation	GitHub Actions
Domain	Any suitable registrar
Analytics	Optional privacy-conscious analytics later

19. Key Design Principles

- Mobile-first: many applicants will use Android phones.
- Privacy-first: process personal images locally whenever possible.
- Official-source-first: requirements should be tied to current official notifications.
- Simple workflow: users should not need to understand pixels or compression quality.
- Transparent validation: explain exactly why a file passes or fails.
- Versioned requirements: retain historical exam specifications.
- Low operating cost: avoid servers until a real need exists.
- Scalable architecture: keep image processing independent from the exam database.

20. Conclusion

A GitHub-first implementation is technically feasible and well suited to this project. The initial platform can be a static web application hosted on GitHub Pages, with image operations performed locally in the browser and exam specifications maintained as version-controlled data. This approach minimizes infrastructure cost, improves privacy and makes the system easy to maintain. Backend services can be introduced later only when features such as user accounts, payments, cloud storage, advanced AI processing or a larger dynamic database become necessary.